Técnicas de Controle de Concorrência

Adaptado dos
materiais da professora Raqueline Penteado

Sumário

- Controle de Concorrência

- Protocolos de Controle de Concorrência
  - Otimistas
    - Técnicas de validação

  - Pessimistas
    - Técnicas
      - Bloqueio
        - Binário
        - Compartilhado/Exclusivo
        - Protocolo de bloqueio em duas fases (2PL)

      - Ordenação por *Timestamp*

Controle de Concorrência

- Técnicas de controle de concorrência podem ser utilizadas para garantir o **isolamento** de transações concorrentes.

- **Protocolos de controle de concorrência**
  - Oferecem **regras** que, se seguidas pelas transações, garantem a **serialização** de todos os escalonamentos nos quais as transações participam.

Protocolos de Controle de Concorrência

- Otimistas e Pessimistas
  - **Otimistas**
    - Baseados na premissa de que **conflitos entre as transações** são **raros**.
    - Testam as transações **após** a execução de suas operações.
      - Técnicas de validação

Protocolo Otimista

- **Técnicas de Validação**
  - São otimistas porque assumem que ocorrerá **pouca interferência** entre as transações.
  - Nenhuma verificação é feita enquanto a transação estiver sendo executada.
  - Ao fim da execução, uma fase de validação verifica se alguma das atualizações de transações violou a serialização.
    - Se não violou, a transação é efetivada e o BD é atualizado.
    - Se violou, a transação é abortada e reiniciada mais tarde.

Protocolo Otimista

- **Técnicas de Validação**
  - A estratégia é realizar todas as verificações de uma só vez.
  - Logo, a execução da transação continua com um mínimo de sobrecarga até que a fase de validação seja alcançada.

Protocolo Otimista

- **Técnica de Validação - Fases**
  - **Leitura**
    - Uma transação pode ler valores de itens de dados **efetivados do BD**.
    - As atualizações são aplicadas apenas à **cópias locais** dos itens de dados, mantidos em um **espaço de trabalho** da transação.
  - **Validação**
    - A verificação é executada para garantir que a serialização não seja violada **se** as atualizações forem aplicadas ao BD.

Protocolo Otimista

- **Técnica de Validação - Fases**
  - **Escrita**
    - Se a fase de validação é **bem-sucedida**, as **atualizações** da transação são **aplicadas** ao BD.
    - Caso contrário, as atualizações são **descartadas** e a transação é **reiniciada**.

Protocolos de Controle de Concorrência

- Otimistas e Pessimistas
  - **Pessimistas**
    - Baseados na premissa de que **conflitos entre as transações ocorrem com frequência**.
    - Testam as transações **antes** da execução de suas operações.
      - Bloqueio em duas fases
        - Solução padrão em SGBD convencionais.
      - Ordenação *por Timestamp*

Técnica de Bloqueio

- Umas das principais técnicas usadas para controle de execução concorrente de transações.
  - Bloqueio de itens de dados.

- Subsistema gerenciador de bloqueio.

- **Bloqueio**
  - Variável associada a um item do BD.
  - Indica o ***status*** do item de dado com relação às possíveis operações que podem ser aplicadas.
  - Geralmente existe um bloqueio associado a **cada** item de dado do BD.
  - São usados como meio de **sincronizar** o acesso por transações concorrentes aos itens do BD.

Técnica de Bloqueio

- **Tipos de Bloqueios**
  - **1 - Bloqueio Binário**
    - Pode assumir dois estados
      - **Bloqueado:** 1
      - **Desbloqueado:** 0
    - Um bloqueio distinto é associado a cada item X do BD.
    - **Valor do bloqueio em X**
      - 1, indica que o item X não pode ser acessado por uma operação de BD que solicite o item.
      - 0, indica que o item X pode ser acessado quando solicitado.

Técnica de Bloqueio

- **Operações**
  - *Lock_item* (item de dado);
  - *Unlock_item* (item de dado);
- Operações são indivisíveis.
- SGBD oferece um subsistema de gerenciamento para manter e controlar o acesso aos bloqueios.
  - Tabela de Bloqueio do Sistema;
  - Itens que estão bloqueados;
  - <item_dado, LOCK, transação de bloqueio> + lista de espera.

Técnica de Bloqueio

```
lock_item(X):
B:   if LOCK(X) = 0                    (* item is unlocked *)
         then LOCK(X) ←1               (* lock the item *)
     else
         begin
         wait (until LOCK(X) = 0
             and the lock manager wakes up the transaction);
         go to B
         end;
unlock_item(X):
     LOCK(X) ← 0;                      (* unlock the item *)
     if any transactions are waiting
         then wakeup one of the waiting transactions;
```

Técnica de Bloqueio

- No bloqueio binário, cada transação deve obedecer diferentes regras.

- **Exemplos**
  - Uma transação T deve garantir a operação ***lock_item*(X)** antes que qualquer operação **ler_item(X)** ou **escrever_item(X)** seja executada em T.
  - Uma transação T deve garantir a operação ***unlock_item*(X)** depois que **todas** as operações **ler_item(X)** e **escrever_item(X)** sejam completadas em T.

Técnica de Bloqueio

- No bloqueio binário, cada transação deve obedecer diferentes regras.

- **Exemplos**
  - Uma transação T não resultará em uma operação ***lock_item*(X)** se ela já tiver o bloqueio no item X.

  - Uma transação T não resultará em uma operação ***unlock_item*(X)**, a menos que ela já tenha o bloqueio no item X.

Técnica de Bloqueio

- **Conclusão**
  - Força exclusão mútua
    - Apenas uma transação por vez pode possuir o bloqueio de um determinado item de dado.
    - Duas transações **não podem** ser acessar o mesmo item de dado **concorrentemente**.
  - Conceito muito restritivo
    - Não utilizado na prática.

Técnica de Bloqueio

- **Tipos de Bloqueios**
  - **2 - Bloqueio Exclusivo e Compartilhado**
    - Compartilhado
      - Diferentes transações podem acessar o mesmo item de dado simultaneamente.
      - Finalidade: **leitura**.
    - Exclusivo
      - Apenas uma transação pode acessar um item de dado em um determinado momento.
      - Finalidade: **escrita**.

Técnica de Bloqueio

- **Valores de Status**
  - Bloqueado para leitura (***read_locked***)
  - Bloqueado para escrita (***write_locked***)
  - Desbloqueado (***unlocked***)

- **Associação ao item de dado**

<item_dado, **tipo_bloqueio**, nro_leitores_do_item>

+

fila de espera para transações que não podem acessar o item de dado requisitado

Técnica de Bloqueio

- Se **bloqueio**(**item_dado**) = ***read_locked***
  - item_dado pode ser acessado desde que a operação que o requisitou seja de leitura.

- Se **bloqueio**(**item_dado**) = ***write_locked***
  - item_dado não pode ser acessado.

- Se **bloqueio**(**item_dado**) = ***unlocked***
  - item_dado pode ser acessado.

- **Operações *read_locked, write_locked* e *unlocked***
  - São **unidades indivisíveis**.

Técnica de Bloqueio

```
read_lock (X):

  B: if LOCK (X)="unlocked"
     then begin LOCK (X)← "read-locked";
                no_of_reads(X)← 1
          end
     else if LOCK(X)="read-locked"
          then no_of_reads(X)← no_of _reads(X) + 1
          else begin wait (until LOCK (X)="unlocked" and
                     the lock manager wakes up the transaction);
                go to B
                end;
```

Técnica de Bloqueio

```
write_lock (X):
  B: if LOCK (X)="unlocked"
     then LOCK (X)← "write-locked";
     else begin
          wait (until LOCK(X)="unlocked" and
            the lock manager wakes up the transaction);
          go to B
          end;
```

Técnica de Bloqueio

```
unlock (X):
  if LOCK (X)="write-locked"
     then begin LOCK (X) ← "unlocked";
          wakeup one of the waiting transactions, if any
          end
     else if LOCK(X)="read-locked"
           then begin
                no_of_reads(X) ← no_of_reads(X) – 1;
                if no_of_reads(X)=0
                  then begin LOCK (X)="unlocked";
                       wakeup one of the waiting transactions, if any
                       end
                  end;
```

Técnica de Bloqueio

- **Regras seguidas pelas transações**
  - Uma transação T deve realizar a operação ***read_lock*(X)** ou ***write_lock*(X)** antes de qualquer operação ***read_item*(X)** em T.
  - Uma transação T deve realizar a operação ***write_lock*(X)** antes de qualquer operação ***write_item*(X)** em T.
  - Uma transação T deve realizar a operação ***unlock*(X)** depois de **todas** as operações ***read_item*(X)** ou ***write_item*(X)** de T.

Técnica de Bloqueio

- **Regras seguidas pelas transações**
  - Uma transação T não realizará uma operação ***read_lock*(X)** se ela já possuir uma trava compartilhada ou exclusiva de X.
  - Uma transação T não realizará uma operação ***write_lock*(X)** se ela já possuir uma trava compartilhada ou exclusiva de X.
  - Uma transação T não realizará uma operação **unlock(X)** se ela não possuir uma trava compartilhada ou exclusiva de X.

Técnica de Bloqueio

- **Conversão de Bloqueio**
  - ***Upgrade*** (**ampliação**)
    - Conversão de bloqueio compartilhado para bloqueio exclusivo.

read_lock(item_dado) $\longrightarrow$ ***write_lock (item_dado)***

  - ***Downgrade*** (**redução**)
    - Conversão do bloqueio exclusivo para bloqueio compartilhado.

write_lock(item_dado) $\longrightarrow$ ***read_lock(item_dado)***

Protocolo de Bloqueio em Duas Fases

- Protocolo de Bloqueio de Duas Fases Básico
  - *Two Phase Locking*- 2PL

- **Explicação**

Técnica de bloqueio compartilhado/exclusivo

+

protocolo adicional

- Protocolo define o **posicionamento das operações** de **bloqueio** e **desbloqueio** em toda a transação.

Protocolo de Bloqueio em Duas Fases

- Uma transação que segue o protocolo 2PL possui duas fases.
  - **Fase de crescimento ou expansão**
    - A transação apenas pode **requisitar** novos bloqueios, mas não pode liberá-los.
  - **Fase de encolhimento**
    - A transação pode apenas **liberar** seus bloqueios, mas não pode requisitar novos bloqueios.

Protocolo de Bloqueio em Duas Fases

- Uma transação T segue o 2PL se
  - **Todas** as operações ***read_lock*** *e* ***write_lock*** de T precedem a primeira operação *unlock* de T.

    - Pode existir uma operação de ***upgrade*** e ***downgrade***.

Protocolo de Bloqueio em Duas Fases

(a)

T_1	T_2
read_lock(*Y*);	read_lock(*X*);
read_item(*Y*);	read_item(*X*);
unlock(*Y*);	unlock(*X*);
write_lock(*X*);	write_lock(*Y*);
read_item(*X*);	read_item(*Y*);
X:=*X*+*Y*;	*Y*:=*X*+*Y*;
write_item(*X*);	write_item(*Y*);
unlock(*X*);	unlock(*Y*);

(b) Valores iniciais: *X*=20, *Y*=30

Resultado do plano de execução serial T_1 seguido por T_2:

X=50, *Y*=80

Resultado do plano serial T_2 seguido por T_1:

X=70, *Y*=50

- **Transações obedecem o bloqueio 2PL?**

Protocolo de Bloqueio em Duas Fases

(c)

T_1	T_2
read_lock(*Y*);	
read_item(*Y*);	
unlock(*Y*);	
	read_lock(*X*);
	read_item(*X*);
	unlock(*X*);
	write_lock(*Y*);
	read_item(*Y*);
	Y:=*X*+*Y*;
	write_item(*Y*);
	unlock(*Y*);
write_lock(*X*);	
read_item(*X*);	
X:=*X*+*Y*;	
write_item(*X*);	
unlock(*X*);	

Tempo ↓

Resultado do plano S:
X=50, *Y*=50
(não serializavel)

- Transações que não obedecem o bloqueio 2PL
- Como poderiam ficar as transações T1 e T2 obedecendo ao protocolo?

Protocolo de Bloqueio em Duas Fases

T_1'	T_2'
read_lock (*Y*);	read_lock (*X*);
read_item (*Y*);	read_item (*X*);
write_lock (*X*);	write_lock (*Y*);
unlock (*Y*);	unlock (*X*);
read_item (*X*);	read_item (*Y*);
X:=*X*+*Y*;	*Y*:=*X*+*Y*;
write_item (*X*);	write_item (*Y*);
unlock (*X*);	unlock (*Y*);

Exemplo

- **Aplique o protocolo 2PL básico e simule uma execução concorrente.**

T1	T2	T3
read_item(b)	read_item(b)	read_item(a)
read_item(a)	b= b – 50	a = a + 100
display(a+b)	write_item(b)	write_item(a)
	read_item(a)	read_item(c)
	a = a + 50	read_item(d)
	write_item(a)	display(a+c+d)
	display(a+b)	

Exemplo

T1	T2	T3
read_lock(b)	read_lock(b)	read_lock(a)
read_item(b)	read_item(b)	read_item(a)
read_lock(a)	write_lock(b)	write_lock(a)
read_item(a)	b= b – 50	a = a + 100
unlock(b)	write_item(b)	write_item(a)
unlock(a)	read_lock(a)	read_lock(c)
display(a+b)	read_item(a)	read_item(c)
	write_lock(a)	read_lock(d)
	a = a + 50	read_item(d)
	write_item(a)	unlock(a)
	unlock(b)	unlock(c)
	unlock(a)	unlock(d)
		display(a+c+d)

Exemplo

T1	T2	T3
read_lock(b)		
read_item(b)		
		read_lock(a)
		read_item(a)
read_lock(a)		
read_item(a)		
unlock(b)		
unlock(a)		
display(a+b)		
		write_lock(a)
		a = a + 100
		write_item(a)
	read_lock(b)	
	read_item(b)	
	write_lock(b)	
	b= b – 50	
	write_item(b)	
		read_lock(c)
		read_item(c)
		read_lock(d)
		read_item(d)
		unlock(a)
		unlock(c)
		unlock(d)
		display(a+c+d)
	read_lock(a)	
	read_item(a)	
	write_lock(a)	
	a = a + 50	
	write_item(a)	
	unlock(b)	
	unlock(a)	

Protocolo de Bloqueio em Duas Fases

- **Variantes Protocolo de Bloqueio em Duas Fases**
  - Protocolo de bloqueio em duas fases **conservador**
    - Cada transação realiza todos os bloqueios **antes** que iniciar.
    - Livre de *deadlock*.
  - Protocolo de bloqueio em duas fases **estrito**
    - **Bloqueios de modo exclusivo** são liberados somente ao final da transação, quando a mesma é confirmada.
      - Não é livre de *deadlock*.

Protocolo Pessimista

- **Impasse** ***(Deadlock)***
  - Quando uma transação T1 espera por um novo bloqueio sendo utilizado por T2, T2 espera por um bloqueio sendo utilizado por Tn e Tn espera por um bloqueio sendo utilizado por T1.

(a)

T_1'	T_2'
read_lock(*Y*);	
read_item(*Y*);	
	read_lock(*X*);
	read_item(*X*);
write_lock(*X*);	
	write_lock(*Y*);

Tempo ↓

Protocolo Pessimista

- **Impasse** ***(Deadlock)***
  - **Simplicidade:** *timeout*
  - **Detecção** de impasse
    - Verifica se um impasse realmente existe.
    - Plano com transações que pouco interferem nas outras.

Protocolo Pessimista

- **Impasse** *(**Deadlock**)*
  - **Simplicidade:** *timeout*
  - **Detecção** de impasse
    - **Grafo de espera**
      - Se uma transação Ti espera por uma transação Tj, criar um **ligação direcionada Ti -> Tj**
      - Quando a espera terminar retirar a ligação do grafo.
      - Se existir um ciclo, existe um impasse.
    - Como selecionar as "**vítimas**"?
      - As que fizeram pouca alteração no BD.

Protocolo Pessimista

- **Inanição *(Starvation)***
  - Quando uma transação T não consegue ser executada por um período de tempo indefinido, enquanto que outras transações continuam suas execuções normalmente.

  - **Possibilidades de solução**
    - Esquema de espera imparcial
      - Fila onde o primeiro que chega é o primeiro a ser servido.
    - Alternativa
      - Aumentar a prioridade de uma transação que foi abortada inúmeras vezes.

Ordenação por *Timestamp*

- O algoritmo deve garantir que, para cada **item acessado** pelas **operações conflitantes** no **plano de execução**, a ordem na qual o item é acessado não viola a **serialização**.

- As transações são **ordenadas** por seus ***timestamps***.

Ordenação por *Timestamp*

- *Timestamp* (**marca de tempo**)
  - Identificador único, criado pelo SGBD, para **identificar uma transação**.
  - Seus valores são designados na **ordem** em que as transações são **submetidas ao sistema**.
  - **Momento de início da transação**

$$TS(T_1) < TS(T_2)$$

Ordenação por *Timestamp*

- **Cada item X do BD possui dois valores**
  - *Read_TS(X): o maior timestamp de leitura do item X*.
  - *Write_TS(X): o maior timestamp de escrita do item X*.

- Protocolo de ordenação por *timestamp*
  - Assegura que quaisquer operações de leitura e escrita sejam executadas por **ordem** de *timestamp*.

Ordenação por *Timestamp*

- **Protocolo** (**Visão Geral**)
  - Suponha que a **transação Ti** emita um **read(Q)**
    - Se **TS(Ti) < Write-TS(Q)**, então Ti precisa ler um valor de Q que já foi sobreposto. Assim, a operação read é rejeitada e Ti é desfeita.

    - Se **TS(Ti) >= Write-TS(Q)**, então a operação read é executada e Read_TS(Q) **recebe o maior valor entre Read_TS(Q) e TS(Ti)**.

Uma transação Ti que foi desfeita pelo esquema de controle de concorrência, decorrente de uma operação read ou write, recebe um novo timestamp e é reiniciada

Ordenação por *Timestamp*

- **Protocolo** (**Visão Geral**)
  - Suponha que a **transação Ti** emita um **write(Q)**
    - Se **TS(Ti) < Read-TS(Q)**, então o valor de Q que Ti está produzindo foi necessário antes e o sistema assumiu que aquele valor nunca seria produzido. Logo, a operação write é rejeitada e Ti é desfeita.
    - Se **TS(Ti) < Write-TS(Q)**, então Ti está tentando escrever um valor obsoleto em Q. Logo, essa operação write é rejeitada e Ti é desfeita.
    - De outro modo, a operação write é executada e Write-TS(Q) é registrado com o valor de TS(Ti).

Ordenação por *Timestamp*

- Considere os seguintes *timestamps:*
  - Escalonamento 1
    - TS(T1) = 6
    - TS(T2) = 1
    - TS(T3) = 4

transação T_1	transação T_2	transação T_3
	ler_item (Z) ler_item (Y) escrever_item (Y);	
		ler_item (Y) ler_item (Z)
ler_item (X) escrever_item (X);		escrever_item (Y); escrever_item (Z);
	ler_item (X);	
ler_item (Y) escrever_item (Y);		
	escrever_item (X);	

Tempo ↓

Ordenação por *Timestamp*

- Considere os seguintes *timestamps:*
  - Escalonamento 2
    - TS(T1) = 3
    - TS(T2) = 7
    - TS(T3) = 1

transação T_1	transação T_2	transação T_3
		ler_item (*Y*) ler_item (*Z*)
ler_item (*X*) escrever_item (*X*);		escrever_item (*Y*); escrever_item (*Z*);
	ler_item (*Z*)	
ler_item (*Y*) escrever_item (*Y*);		
	ler_item (*Y*) escrever_item (*Y*); ler_item (*X*); escrever_item (*X*);	

Tempo ↓

Considerações Finais

- Neste material, os principais **protocolos de controle de concorrência** foram apresentados.

- Os protocolos podem ser classificados em **otimistas** e **pessimistas**.

- Como **exemplo** de **protocolo otimista**, considerou a **validação**.

- Como **exemplos** de **protocolos pessimistas**, considerou-se os bloqueios **binário**, **exclusivo/compartilhado**, **em duas fases** e **ordenação por *timestamp***.

Referências

- Sistemas de Banco de Dados. Elsmari e Navathe. Pearson-Addison Wesley, Sexta Edição.

- Sistemas de Banco de Dados. Silberchatz, Korth e Sudarshan. Makron Books, Terceira Edição.

- Introdução a Sistemas de Bancos de Dados. Date. Campus. Tradução da Oitava Edição Americana.